

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 2**

**COURSE NAME:** Artificial Intelligence

**COURSE CODE:** CSA17

---

### **COURSE OUTCOME COVERED**

**CO2:** Experiment search and game-solving techniques such as Greedy Search, A\*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

*Assessment Tool Weightage: 50%*

---

**QUESTIONS: 5**

**Duration: 1 Hour**  
**Total Marks:50**

Annexure A

### **Scenario Based Assignment**

---

#### ***Code of Conduct:***

*I \_\_\_\_\_ K. Deeksha Sree(192424097)\_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: K. Deeksha Sree***

## Annexure A

### Scenario Based Assignment

**1. Scenario :** A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A\* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

**2. Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

**Tasks.** Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

**3. Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search

- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

## RUBRICS FOR CALCULATION

| Criteria                     | Weightage (%) | Level 4<br>(Exemplary)                                                    | Level 3<br>(Proficient)                            | Level 2<br>(Developing)                           | Level 1<br>(Beginning)                            |
|------------------------------|---------------|---------------------------------------------------------------------------|----------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| Understanding of Scenario    | 20%           | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps           | Partial understanding; misses key aspects         | Misinterprets or fails to understand the scenario |
| Identification of Key Issues | 20%           | Accurately identifies all relevant issues and factors involved            | Identifies most key issues with minor omissions    | Identifies limited issues; some irrelevant points | Unable to identify key issues                     |
| Application of Concepts      | 25%           | Applies appropriate concepts effectively to address the scenario          | Applies concepts with minor errors                 | Limited or partially correct application          | Unable to apply relevant concepts                 |
| Decision Making              | 20%           | Makes well-reasoned, logical decisions with strong rationale              | Makes reasonable decisions with some justification | Makes weak decisions with limited justification   | No clear decisions made or rationale provided     |
| Clarity of Response          | 15%           | Response is clear, structured, and logically presented                    | Generally clear with minor issues                  | Somewhat unclear or poorly structured             | Disorganized and difficult to understand          |

### 1.(i) Greedy Best-First Search (GBFS)

How it works:

- Uses only the heuristic value  $h(n)$  (straight-line distance).
- Chooses the path that appears closest to the destination.

Strengths:

- Very fast.
- Explores fewer nodes.
- Suitable for urgent situations.

Weaknesses:

- Does not guarantee the shortest or safest path.
- Ignores actual travel cost (weather, terrain, battery).
- Performs poorly if paths become blocked.

### (ii) *A Search\**

How it works:

- Uses the evaluation function:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$  = actual cost from the start.
- $h(n)$  = estimated cost to the goal.

Advantages:

- Finds the optimal path (with an admissible heuristic).
- Considers both travel cost and distance.
- Adapts better to changing conditions.

Disadvantages:

- Uses more memory.
- Slower than GBFS.

### (iii) Impact of Heuristic Accuracy

- Accurate heuristic:
  - GBFS becomes faster.
  - $A^*$  finds the optimal path efficiently.
- Inaccurate heuristic:
  - GBFS may choose poor or blocked routes.
  - $A^*$  still performs well because it also considers the actual path cost.

### (iv) Effect of Dynamic Changes (Blocked Paths)

- Floods or road blockages require the route to be updated.
- GBFS: May need to restart the search and can waste time.
- $A^*$ : Updates path costs and finds an alternative route more effectively.

### (v) Recommended Algorithm

Recommended: *A Search\**

Justification:

- Finds the safest and shortest route.
- Considers travel cost, weather, and terrain.
- Adapts to blocked paths.

- More reliable for emergency medicine delivery, even though it is slightly slower than GBFS.

#### Comparison

| Feature                 | GBFS      | A*       |
|-------------------------|-----------|----------|
| Uses Heuristic          | ✓         | ✓        |
| Uses Actual Cost        | ✗         | ✓        |
| Speed                   | Very Fast | Moderate |
| Optimal Path            | ✗         | ✓        |
| Handles Dynamic Changes | Poor      | Good     |
| Best for this Scenario  | ✗         | ✓        |

2. A smart city uses an AI system to optimize traffic signals at hundreds of intersections. The goals are to reduce waiting time, fuel consumption, and traffic congestion. The search space is very large, traffic changes continuously, and the system may get stuck in locally optimal solutions.

#### Formulate the Problem

This is an **optimization problem** because the system must find the best signal timings that minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

#### Why is Traditional Search Inefficient?

- The number of possible signal combinations is extremely large.
- Traffic conditions change continuously.
- Searching every possible solution takes too much time.

#### (i) Hill Climbing

##### How it Works

- Starts with an initial traffic signal timing.
- Makes small changes to improve traffic flow.
- Chooses the neighboring solution with the best improvement.
- Stops when no better solution is found.

##### Limitations

- May get stuck at a local optimum.
- Cannot always find the global optimum.
- Performs poorly in dynamic traffic conditions.

#### (ii) Local Maxima, Plateaus, and Ridges

##### Local Maxima

- A solution better than nearby solutions but not the best overall.
- **Example:** Traffic improves in one area but congestion increases elsewhere.

##### Plateau

- Many neighboring solutions have the same value.
- **Example:** Changing signal timings produces no noticeable improvement.

##### Ridge

- The best solution requires multiple changes together.

- **Example:** Adjusting one signal alone is ineffective, but adjusting several intersections simultaneously improves traffic.

### (iii) Simulated Annealing and Genetic Algorithm

#### Simulated Annealing

- Sometimes accepts worse solutions to escape local maxima.
- Gradually reduces random exploration.
- More likely to find the global optimum than Hill Climbing.

#### Genetic Algorithm (GA)

- Treats each signal timing plan as a chromosome.
- Uses **selection, crossover, and mutation** to generate better solutions.
- Searches many solutions simultaneously.
- Adapts well to changing traffic conditions.

### (iv) Recommended Approach

#### Recommended: Genetic Algorithm

##### Justification

- Handles very large search spaces.
- Finds near-optimal solutions.
- Adapts to changing traffic patterns.
- Avoids local optima better than Hill Climbing.
- Suitable for large-scale smart city traffic management.

#### Comparison

| Feature                    | Hill Climbing | Simulated Annealing | Genetic Algorithm |
|----------------------------|---------------|---------------------|-------------------|
| Avoids Local Maxima        | ✗             | ✓                   | ✓                 |
| Handles Large Search Space | ✗             | Moderate            | ✓                 |
| Adaptable                  | Low           | Medium              | High              |
| Scalability                | Low           | Medium              | High              |
| Best Choice                | ✗             | Good                | ✓                 |

**3. An autonomous rover explores Mars to collect samples while avoiding hazards such as rocks and craters. It has limited sensing capability, no complete map, and updates its knowledge as it explores.**

#### (i) Why does this problem require an Online Search Agent?

- The rover does not have a complete map of Mars.
- It must make decisions while exploring the environment.
- It continuously learns from new information.
- Offline search is unsuitable because it requires complete knowledge before starting.

#### (ii) Challenges of Partial Observability and Dynamic Environment

##### Partial Observability

- The rover can only sense nearby areas.
- Hidden obstacles may not be detected until it gets closer.
- It has incomplete information about the terrain.

##### Dynamic Environment

- Dust storms may reduce visibility.

- New obstacles can appear.
- Terrain conditions may change over time.

### (iii) How the Rover Builds and Updates Its Internal Model

- Collects data using cameras and sensors.
- Creates a map of explored areas.
- Marks safe and hazardous locations.
- Updates the map whenever new information is received.
- Uses the updated map to plan future movements.

### (iv) Comparison of Online Search with Uninformed and Informed Search

| Feature                          | Online Search | Uninformed Search | Informed Search |
|----------------------------------|---------------|-------------------|-----------------|
| Complete Map Required            | ✗ No          | ✓ Yes             | ✓ Yes           |
| Uses Heuristic                   | Sometimes     | ✗ No              | ✓ Yes           |
| Learns While Exploring           | ✓ Yes         | ✗ No              | ✗ No            |
| Suitable for Unknown Environment | ✓ Yes         | ✗ No              | ✗ No            |

### (v) Recommended Approach

#### Recommended: Online Search Agent

##### Justification

- Works without a complete map.
- Adapts to new obstacles and changing conditions.
- Continuously updates its knowledge.
- Ensures safer navigation in unknown environments.
- Best suited for real-time decision making on Mars.

**4. A university wants an AI system to generate exam timetables while satisfying constraints such as no overlapping exams, limited exam halls, faculty availability, subject order, and special accommodations.**

### Formulate the Problem as a CSP

A Constraint Satisfaction Problem (CSP) consists of:

- **Variables**
- **Domains**
- **Constraints**

### (i) Variables, Domains, and Constraints

#### Variables

These are the exams to be scheduled.

- Example: **Math, Physics, Chemistry, English**

#### Domains

Possible values for each variable.

- Time slots (Morning, Afternoon, Evening)
- Exam halls (Hall 1, Hall 2, Hall 3)

#### Constraints

- No student should have two exams at the same time.
- Limited number of exam halls.
- Some subjects must be scheduled before others.
- Faculty must be available during the exam.



- Students with special accommodations should receive suitable arrangements.

## **(ii) Backtracking Search**

### **How it Works**

- Assign a time slot to one exam.
- Check whether all constraints are satisfied.
- If a conflict occurs, go back (backtrack) and choose another time slot.
- Continue until all exams are scheduled successfully.

### **Advantage**

- Guarantees a valid timetable if one exists.

## **(iii) Constraint Propagation (Forward Checking)**

### **How it Works**

- After assigning a time slot to an exam, remove invalid choices for the remaining exams.
- Detects conflicts early.
- Reduces unnecessary searching.

### **Benefits**

- Faster timetable generation.
- Fewer backtracking steps.
- Improves efficiency for large problems.

## **(iv) Real-World Challenges and Best Strategy**

### **Challenges**

- Large number of courses and students.
- Last-minute faculty unavailability.
- Limited exam halls.
- Special accommodation requirements.
- Changes in the timetable.

### **Best Strategy**

Use **Backtracking with Forward Checking (Constraint Propagation)**.

### **Justification**

- Produces valid timetables.
- Detects conflicts early.
- Reduces computation time.
- Scales well for large universities.

## **Summary Table**

| <b>Component</b> | <b>Description</b>                                                                   |
|------------------|--------------------------------------------------------------------------------------|
| Variables        | Exams to be scheduled                                                                |
| Domains          | Time slots and exam halls                                                            |
| Constraints      | No overlap, hall limits, faculty availability, subject order, special accommodations |
| Search Method    | Backtracking                                                                         |
| Optimization     | Forward Checking (Constraint Propagation)                                            |
| Best Approach    | Backtracking + Forward Checking                                                      |

**5. A gaming company is developing an AI opponent for a real-time strategy game. The AI must make quick decisions, evaluate many possible moves, and respond within strict time limits.**

**(i) Explain how the Minimax Algorithm models this problem**

**How it Works**

- Minimax is used in **two-player games**.
- The AI (**MAX**) tries to maximize its score.
- The opponent (**MIN**) tries to minimize the AI's score.
- The algorithm explores all possible moves and selects the best one.

**Advantage**

- Helps the AI choose the best possible move assuming the opponent also plays optimally.

**(ii) Computational Challenges of Large Game Trees**

**Challenges**

- A large number of possible moves creates a huge game tree.
- Evaluating every move takes a lot of time and memory.
- Real-time games require quick decisions, making full Minimax impractical.

**(iii) How Alpha-Beta Pruning Improves Efficiency**

**How it Works**

- Alpha-Beta Pruning removes branches that **cannot affect the final decision**.
- It avoids evaluating unnecessary game states.

**Benefits**

- Reduces the number of nodes explored.
- Saves time and memory.
- Produces the **same optimal result** as Minimax but much faster.

**(iv) Design an Evaluation Function**

The evaluation function assigns a score to each game state.

**Factors Considered**

- Number of AI units remaining.
- Number of opponent units remaining.
- Health of units.
- Resources collected.
- Control of important locations.
- Distance to the objective.

**Example:**

**Evaluation Score = Resources + Unit Strength + Territory Control – Opponent Strength**

A higher score indicates a better position for the AI.

**(v) Recommend the Best Strategy**

**Recommended: Minimax with Alpha-Beta Pruning**

**Justification**

- Finds the best move.
- Reduces unnecessary computations.
- Meets real-time decision-making requirements.
- Handles large game trees efficiently.
- Provides a balance between **optimality** and **speed**.

## Comparison

| Feature                      | Minimax  | Alpha-Beta Pruning |
|------------------------------|----------|--------------------|
| Finds Best Move              | ✓        | ✓                  |
| Explores All Nodes           | ✓        | ✗                  |
| Faster                       | ✗        | ✓                  |
| Uses Less Memory             | ✗        | ✓                  |
| Suitable for Real-Time Games | Moderate | ✓                  |